D Y PATIL AGRICULTURE & TECHNICAL UNIVERSITY
TALSANDE

MCA PLAIN

Project Name:

MedSafe AI-AI-driven medical safety assistant

Team Member:

Smeeta Gawade

Shraddha Satape

Sanika Patil

Saurabh Sawant

❖ Pre-requisites:

Description-

Before using **MedSafe AI**, ensure that your development environment is properly configured. The following components and tools are required to run the application smoothly and enable all AI-powered features.

1. Python Environment Setup

MedSafe AI is developed using **Python 3.10+**, leveraging libraries for web deployment, OCR processing, fuzzy matching, rule-based logic, and AI model interaction.

● Create and activate a dedicated virtual environment (e.g., medsafe_env) to isolate dependencies and maintain a clean setup.

● Official Python documentation: https://www.python.org/downloads/

Figure 2: virtual environment workflow illustration

2. Streamlit Installation & Configuration

Streamlit powers the interactive front-end dashboard of MedSafe AI, enabling tab-based navigation, real-time analysis, and user-friendly visual outputs.

● Install Streamlit via pip:

pip install streamlit

● Streamlit documentation: https://docs.streamlit.io/library/get-started/installation ● Introductory tutorial: https://www.youtube.com/watch?v=JwSS70SZdyM

Figure 3: Streamlit UI

3. OCR Engine Setup (Tesseract OCR)

MedSafe AI uses **Tesseract OCR** to extract text from uploaded prescription images. Proper installation and configuration are required.

● Download and install Tesseract OCR for your operating system

● Official repository: https://github.com/tesseract-ocr/tesseract

● Ensure the Tesseract executable path is correctly configured in the code

Figure 4: OCR Workflow

4. Library Installation & Core Dependencies

MedSafe AI relies on several Python libraries for text extraction, fuzzy matching, AI interaction, and UI rendering. All dependencies can be installed using a requirements.txt file.

Key libraries include:

● streamlit – Interactive web interface

● pytesseract – OCR text extraction

● Pillow (PIL) – Image loading and processing

● rapidfuzz – Fuzzy string matching for medicine detection

● ollama – Local LLM interaction (LLaMA 3)

● json, datetime – Structured data handling and logging

5. LLM Runtime Setup (Ollama)

MedSafe AI uses **LLaMA 3 via Ollama** for AI-powered summarization, extraction, and explanation tasks.

● Install Ollama on your system

● Pull the required model (e.g., llama3)

● Ollama documentation: https://ollama.com

6. Development Environment

A modern IDE is recommended for efficient development, testing, and debugging of the MedSafe AI codebase.

Recommended IDEs:

● Visual Studio Code: https://code.visualstudio.com/

❖ Workflow:

Description-

1. Environment Setup and Dependency Configuration

This phase focuses on preparing a stable and isolated development environment for MedSafe AI and validating the integration of all foundational components.

• **Activity 1.1:** Create and activate a Python virtual environment (e.g., medsafe_env) and install all required dependencies, including streamlit, pytesseract, Pillow, rapidfuzz, ollama, and supporting standard libraries. This ensures dependency isolation and consistent execution across systems.

• **Activity 1.2:** Organize the project structure into modular components, such as medicine database management, OCR processing, fuzzy matching logic, AI prompt handlers, risk scoring utilities, and UI layout configuration. This modularization improves maintainability and scalability.

• **Activity 1.3:** Initialize the Streamlit application and validate seamless communication between the user interface and backend logic, ensuring that user inputs correctly trigger OCR, interaction checks, AI inference, and result rendering.

2. Core Logic Development (Medicine Safety & AI Reasoning Engine)

• **Activity 2.1:** This phase implements the core intelligence behind MedSafe AI, combining deterministic safety rules with controlled generative AI outputs.

Develop the Medicine Interaction Checker, enabling accurate medicine identification using fuzzy matching and cross-referencing with the curated medicine database to detect known interaction warnings.

• **Activity 2.2**: Implement the Prescription OCR and AI Parsing Module, which extracts raw text from prescription images and converts it into structured medicine and drug-salt information using large language models.

• **Activity 2.3:** Build the Symptom Analysis, Side-Effect Monitoring, and Risk Scoring Engine, integrating rule-based logic for emergency risk calculation with AI-generated educational explanations to maintain clarity and transparency.

3. Streamlit UI Implementation and User Interaction

This phase focuses on delivering an intuitive, interactive, and user-friendly healthcare safety interface.

• **Activity 3.1:** Design a multi-tab Streamlit layout covering medicine interaction checks, prescription OCR, symptom guidance, side-effect monitoring, and emergency risk prediction, with real-time updates and clear visual feedback.

• **Activity 3.2:** Configure robust user input handling for text, images, demographic details, and symptom descriptions, while managing session state to preserve results across interactions.

• **Activity 3.3**: Display structured outputs, AI-generated explanations, warnings, metrics, and expandable raw data sections in a clear and interpretable manner to support informed user understanding.

4. Testing, Optimization, and Deployment

This final phase ensures system reliability, performance, and readiness for real-world usage.

• **Activity 4.1:** Test all UI components and backend workflows, verifying accurate medicine detection, OCR extraction, AI responses, and risk scoring behavior across multiple scenarios.

• **Activity 4.2**: Validate safety logic, interaction detection accuracy, and consistency of AI-generated outputs to ensure educational reliability and non-diagnostic compliance.

• **Activity 4.3**: Optimize overall performance, conduct end-to-end testing, and prepare the application for deployment in academic, research, or prototype healthcare settings.

❖ MILESTONE 1: Environment Setup and Dependency Configuration

Activity 1.1: Python Environment and Dependency Installation

Description-

• Create and activate a dedicated virtual environment for MedSafe AI (e.g., medsafe_env) to ensure dependency isolation and reproducible execution.

• Install all required dependencies listed in requirements.txt, including: streamlit, pytesseract, Pillow, rapidfuzz, ollama, and supporting Python libraries.

• Verify compatibility with Python 3.10+ and confirm successful installation of all modules.

• Test library imports and ensure that core components—OCR processing, fuzzy matching, AI model interaction, and UI rendering—operate correctly without runtime errors.

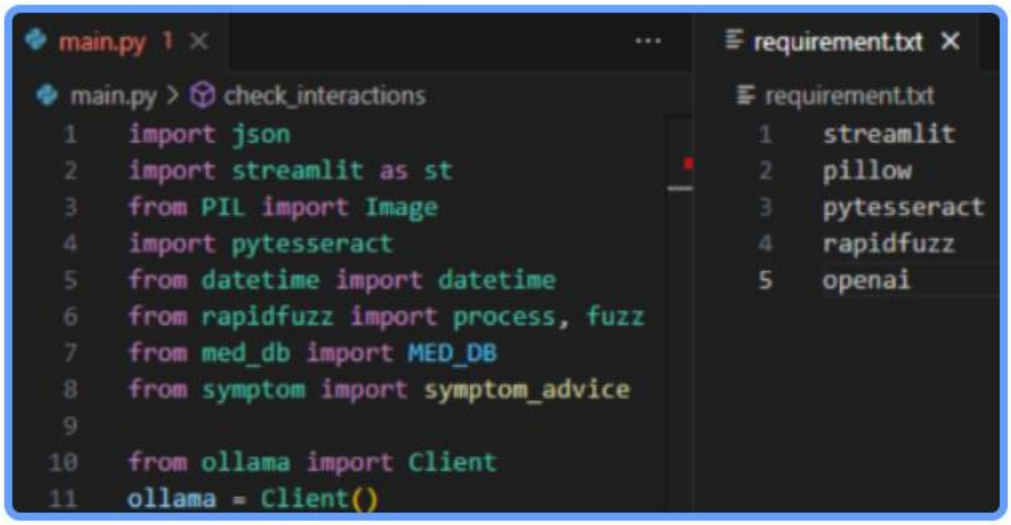


Figure 5: Required & Imported Libraries

Activity 1.2: Project Structure Initialization

Description-

Organize the project into a clear and maintainable structure using modular Python files to separate concerns and improve readability. Key components include:

- streamlit_app.py – Front-end interface and main application logic
- med_db.py – Medicine database and interaction metadata
- symptom.py – Rule-based symptom advice logic
- ocr_utils.py – Prescription OCR and text extraction utilities
- risk_engine.py – Emergency risk scoring and safety rules

• This modular organization ensures scalability, easier debugging, and cleaner integration between deterministic logic and AI-driven components.

Figure 6: Components and Integration Illustration

Activity 1.3: Streamlit Application Initialization

Description-

Launch the application using the command:

streamlit run app.py

• Streamlit ensures consistent theming, responsive layout, and real-time rendering of all MedSafe AI components across devices.

• Validate integration across all modules by testing medicine interaction checks, prescription image uploads, symptom analysis, and risk score visualization

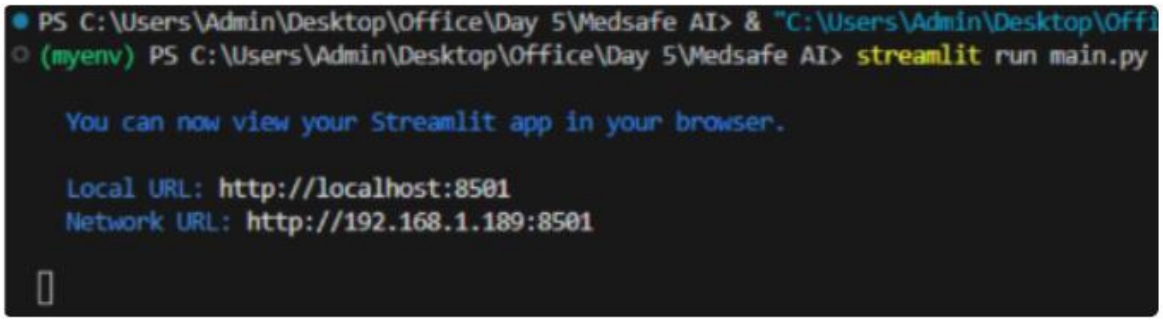


Figure 7: Terminal Illustration

❖ MILESTONE 2: Core Logic Development (Medicine Safety & AI Reasoning Engine)

Activity 2.1: Medicine Interaction & Identification Module Development

Description-

• Implement the medicine identification logic using fuzzy string matching to accurately recognize medicine names from user input and OCR-extracted text, even in the presence of spelling variations or noise.

• Develop the interaction-checking workflow to cross-reference identified medicines against the curated medicine database and detect known drug–drug interaction warnings.

• Generate concise, AI-assisted safety summaries for detected interactions while ensuring non-diagnostic and educational output.

```
def extract_medicines_from_image(img):
    text = pytesseract.image_to_string(img)
    found = []

    for word in text.split():
        med = find_medicine(word)
        if med:
            found.append(med)

    return list(set(found)), text
```

Figure 8: Logic Illustration

Activity 2.2: Prescription OCR and AI-Based Extraction Engine

Description-

• Integrate Tesseract OCR to extract raw textual data from uploaded prescription images.

• Implement LLM-driven parsing to identify medicines and their active drug or salt components, enforcing strict JSON output for structured and reliable data representation.

• Validate extracted data against the medicine database to improve accuracy and downstream safety analysis.

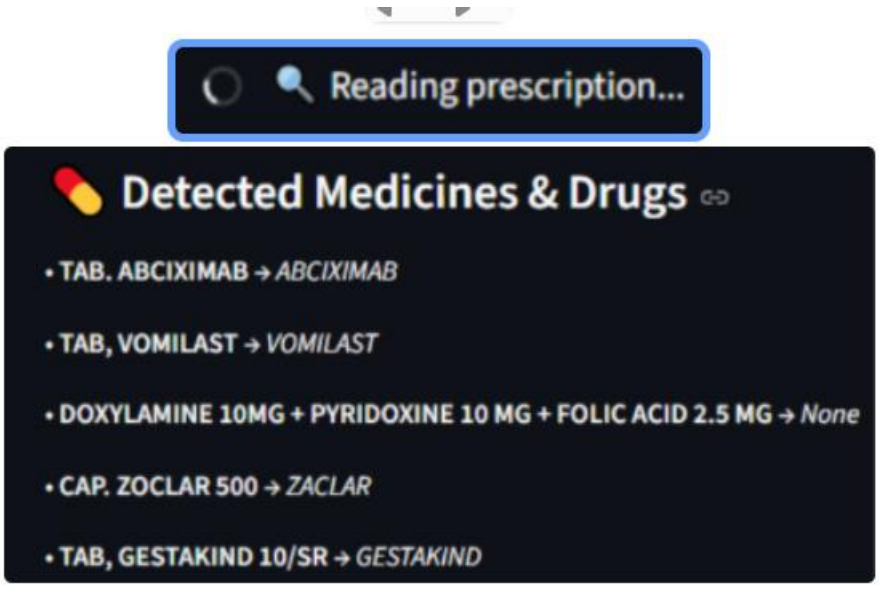


Figure 9: Detected prescription text Illustration

Activity 2.3: Symptom Interpretation, Side-Effect Analysis, and Risk Scoring Engine

Description-

• Build the symptom interpretation module to provide basic rule-based guidance, enhanced by AI-generated educational explanations including home remedies, lifestyle suggestions, and warning signs.

• Implement the side-effect monitoring logic to analyze user-reported experiences in the context of age, gender, medicines, and dosage.

• Develop the emergency risk scoring engine to compute transparent risk percentages based on predefined safety rules and known high-risk indicators.

• Integrate all core modules to ensure a smooth workflow from user input → analysis → AI explanation → UI presentation.

• Implement helper utilities for data handling, session management, and logging to maintain clean and traceable execution.

❖ MILESTONE 3: Streamlit UI Implementation & User Interaction

Activity 3.1: User Interface and Multi-Tab Layout Design

Description-

• Design a structured multi-tab user interface to clearly separate core functionalities, including Medicine Interaction Checker, Prescription OCR, Symptom & Doubt Solver, Side-Effect Monitor, and Emergency Risk Predictor.

• Apply consistent styling and layout using Streamlit components to maintain professional appearance, readable typography, proper spacing, and clear visual hierarchy.

• Ensure responsiveness across different screen sizes and devices, enabling smooth interaction on desktops and tablets.

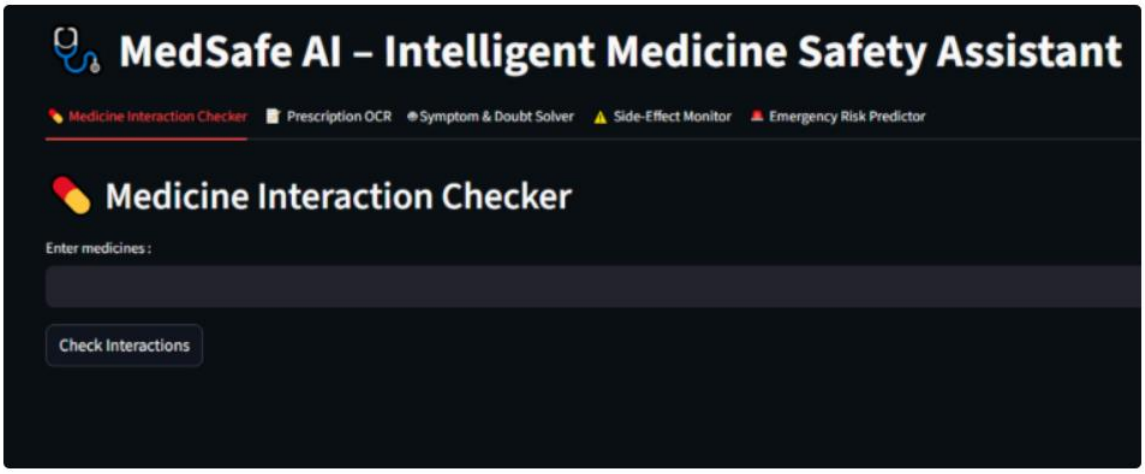


Figure 10: Dashboard Illustration

Activity 3.2: Input Configuration and Session State Management

Description-

- Implement interactive input components such as text fields, text areas, file uploaders, dropdowns, number inputs, and action buttons for capturing user data.
- Use Streamlit session state to persist extracted prescription data, interaction results, side-effect logs, and risk analysis outputs during tab navigation.
- Validate user inputs and implement basic error handling to manage missing, incomplete, or invalid entries gracefully.

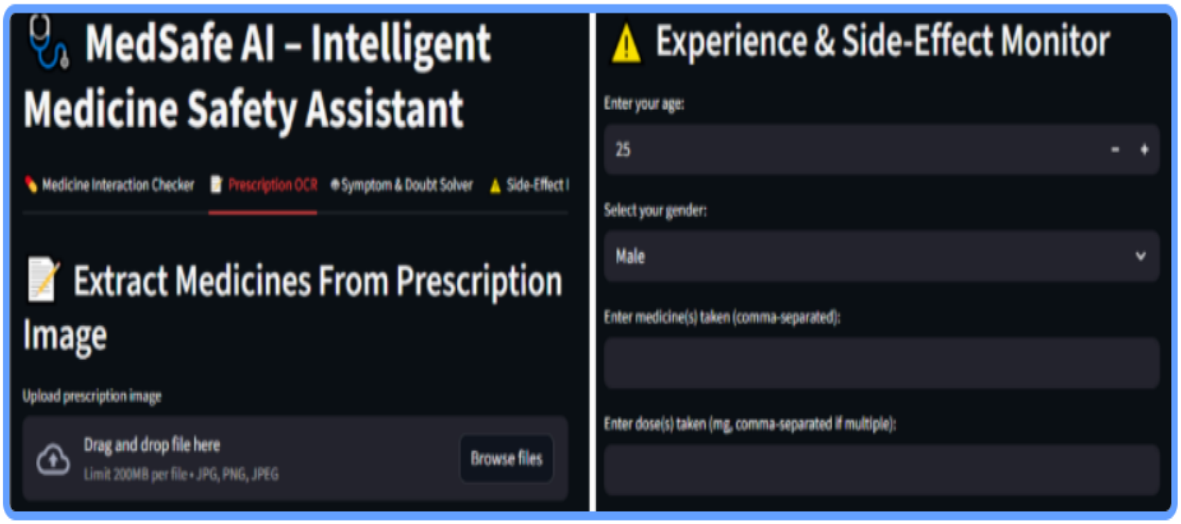


Figure 11: Interactive input components Illustration

Activity 3.3: Output Rendering and Data Visualization

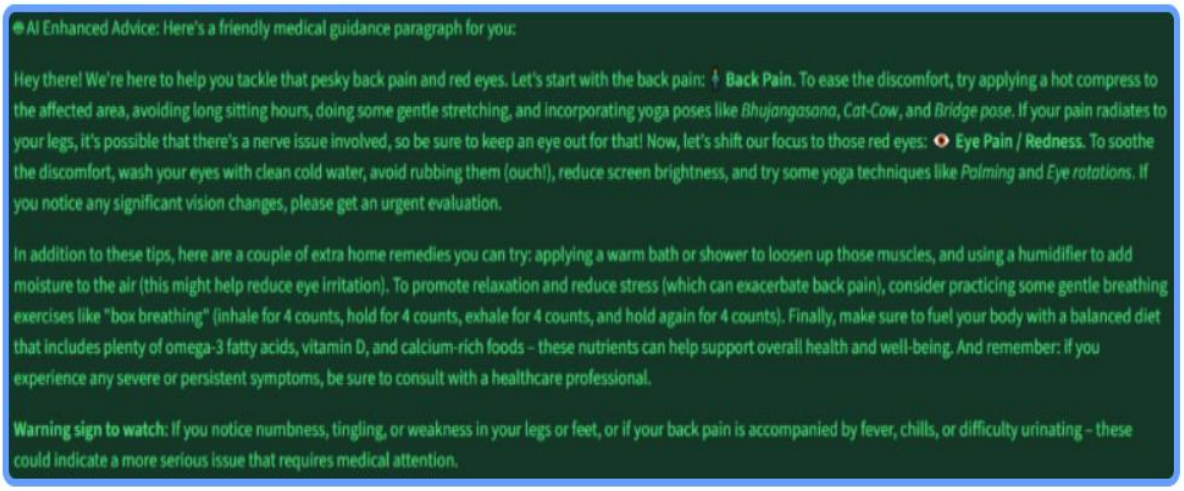
Description-

• Display extracted medicines, interaction warnings, symptom guidance, and AI-generated explanations in a structured and readable format using alerts, metrics, and expandable sections.

• Render emergency risk scores using visual indicators and clearly labeled severity levels to support rapid understanding.

• Present raw OCR text and structured JSON outputs where appropriate to ensure transparency and traceability of AI-assisted results.

• Provide real-time feedback through color-coded messages (success, warning, error) to guide users safely through the analysis process.



AI Enhanced Advice: Here's a friendly medical guidance paragraph for you:

Hey there! We're here to help you tackle that pesky back pain and red eyes. Let's start with the back pain: **Back Pain.** To ease the discomfort, try applying a hot compress to the affected area, avoiding long sitting hours, doing some gentle stretching, and incorporating yoga poses like *Bhujangasana*, *Cat-Cow*, and *Bridge pose*. If your pain radiates to your legs, it's possible that there's a nerve issue involved, so be sure to keep an eye out for that! Now, let's shift our focus to those red eyes: **Eye Pain / Redness.** To soothe the discomfort, wash your eyes with clean cold water, avoid rubbing them (ouch!), reduce screen brightness, and try some yoga techniques like *Palming* and *Eye rotations*. If you notice any significant vision changes, please get an urgent evaluation.

In addition to these tips, here are a couple of extra home remedies you can try: applying a warm bath or shower to loosen up those muscles, and using a humidifier to add moisture to the air (this might help reduce eye irritation). To promote relaxation and reduce stress (which can exacerbate back pain), consider practicing some gentle breathing exercises like "box breathing" (inhale for 4 counts, hold for 4 counts, exhale for 4 counts, and hold again for 4 counts). Finally, make sure to fuel your body with a balanced diet that includes plenty of omega-3 fatty acids, vitamin D, and calcium-rich foods – these nutrients can help support overall health and well-being. And remember: if you experience any severe or persistent symptoms, be sure to consult with a healthcare professional.

Warning sign to watch: If you notice numbness, tingling, or weakness in your legs or feet, or if your back pain is accompanied by fever, chills, or difficulty urinating – these could indicate a more serious issue that requires medical attention.

Figure 12: AI Response to input medical symptoms

❖ MILESTONE 4: Testing, Optimization, and Deployment

Activity 4.1: Functional Testing and Module Verification

Description-

• Test all user interface components, including text inputs, file uploads, buttons, tabs, and alerts, to verify correct behavior across all modules.
• Validate medicine interaction detection, prescription OCR extraction, symptom guidance responses, side-effect analysis, and emergency risk scoring for logical correctness and consistency.
• Ensure AI-generated outputs remain educational, non-diagnostic, and aligned with safety guidelines across multiple test scenarios.

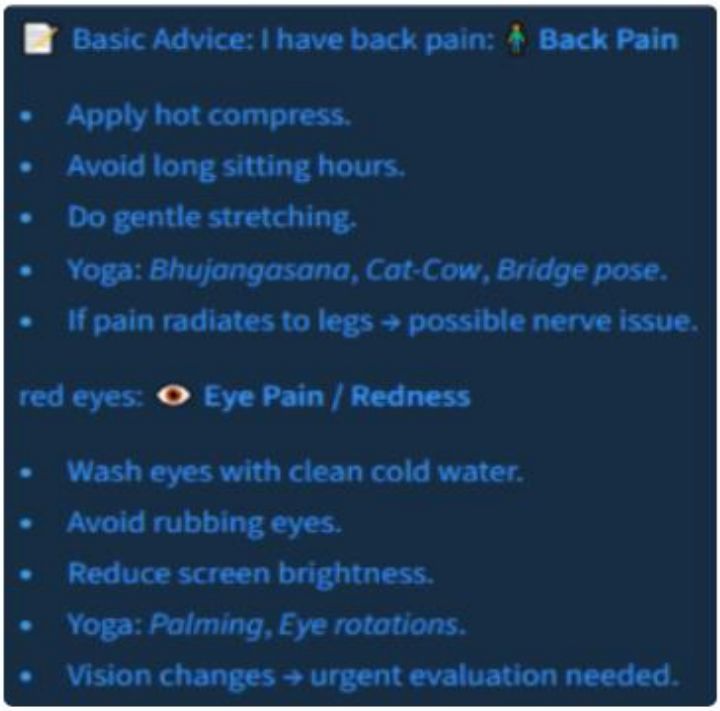


Figure 13: Testing on basic symptoms

Activity 4.2: Performance Testing and Optimization

Description-

• Measure application responsiveness and runtime performance for OCR processing, fuzzy matching, AI inference, and risk score calculation under varying input sizes. • Optimize performance by minimizing redundant OCR calls, improving fuzzy matching efficiency, and managing AI model invocation frequency.

• Test system stability with multiple sequential requests, large symptom descriptions, and complex medicine combinations to ensure reliability under realistic usage conditions.

```
def find_medicine(name):
    if not name:
        return None

    cleaned = name.lower().replace("+", " ").replace(",", " ").replace(".", " ").strip()
    names = list(MED_DB.keys())
    match, score, _ = process.extractOne(cleaned, names, scorer=fuzz.WRatio)
    if score >= 80:
        return match
    return None
```

Figure 14: Logic behind OCR

Activity 4.3: Deployment Preparation and Final Validation

Description-

• Prepare the application for deployment on supported platforms such as local servers or Streamlit Cloud, ensuring environment variables and model dependencies are correctly configured.

• Conduct final end-to-end validation covering user input → analysis → AI explanation → UI output for all major workflows.

• Verify project structure, logging of side-effect experiences, error handling, and safe fallback mechanisms before release.

❖ Conclusion-

MedSafe AI represents a meaningful advancement in AI-assisted healthcare safety and preventive awareness, transforming how users understand medicines, prescriptions, symptoms, and potential health risks through a single intelligent platform. By integrating OCR-based prescription analysis, fuzzy medicine identification, rule-based safety logic, and controlled large language model reasoning, the system delivers clear, structured, and educational insights that support safer decision-making without replacing professional medical consultation. Built on an intuitive Streamlit interface, MedSafe AI provides a seamless and accessible user experience. Users can easily check medicine interactions, extract drug information from prescription images, describe symptoms, monitor side effects, and assess emergency risk levels—all within a clean, responsive dashboard.

The combination of deterministic safety rules and AI-generated explanations ensures transparency, consistency, and interpretability across all outputs. Looking ahead, MedSafe AI holds strong potential for future expansion. Enhancements such as a larger medicine knowledge base, personalized risk profiling, multilingual support, and deeper integration with wearable or health-monitoring systems could further strengthen its impact. Ultimately, this project demonstrates how responsibly designed AI systems can enhance health literacy, promote early risk awareness, and empower individuals with reliable, user-friendly tools for medicine safety and preventive healthcare education.